Final Project Sprint 5:
CS 6440

Valentina Cano Arcay, Lana P kareem, Faezeh Goli
varcay3@gatech.edu, lkareem6@gatech.edu, faezeh@gatech.edu

1 ACCOMPLISHMENTS

1.1 Faezeh

- Completing the OpenFDA Integration and Local Testing (4 hours):

Over the last two weeks, I fully completed the OpenFDA integration on the backend. I created a dedicated router file specifically for OpenFDA requests so our backend structure stays clean and modular. I also wrote the logic for properly formatting API requests, managing API keys, and parsing different OpenFDA JSON response structures. Since OpenFDA can return slightly different fields depending on the drug, I added multiple layers of error handling to keep the integration stable. By the end of this process, the OpenFDA feature was fully functional, and ready to merge into the main branch.

After completing the implementation, I spent time testing everything locally to ensure the routes and OpenFDA responses worked reliably. I tested several medications and checked whether the API returned consistent results. Once the integration behaved as expected across multiple inputs, I felt confident that the feature was ready to be merged.

- Merge OpenFDA integration to Main Branch and Updating the Repository (1 hour):

Once local testing was complete, I merged my OpenFDA branch into the main branch. After merging, I ran another round of tests to make sure nothing broke during the merge. I then pushed everything to our GitHub repository, which officially completed the OpenFDA feature and made the new endpoints accessible to the entire team.

- Debugging the Jose Library Error (1.5 hour):

During development, I encountered a blocking issue related to the python jose library (used for JWT handling). This error prevented the backend from starting correctly and required debugging before I could continue with the project.

- Researching Drug-Drug Interaction (DDI) Data Sources (3 hours):

A significant part of the last two weeks involved researching options for drug-drug interaction data. I evaluated each dataset and API individually to understand their advantages, limitations, and compatibility with our project. The SNAP dataset was promising because it is free and academic friendly, but it uses DrugBank identifiers instead of standard drug names, which makes integration difficult. DrugBank offered excellent clinical quality data, but it requires an academic license, so I submitted an application and am currently waiting for approval. I also explored a Kaggle DDI dataset that is easy to download and inspect. However, it primarily uses generic names rather than brand names. I reviewed the Guide to Pharmacology dataset as well, but it focuses more on drug-target interactions rather than drug-drug interactions. Lastly, I looked into RxNav and RxNav-in-a-Box, but their DDI API is discontinued and the local version is not fully up to date. Overall, this research helped us identify what is realistically usable for our project within our timeline.

- Selecting the Kaggle Dataset and Planning Integration (10 hours):

After reviewing all viable options, my teammates and I decided to move forward with the Kaggle dataset for our prototype because it is the most accessible and requires no licensing approval. I then created a plan for integrating it into our application. This included downloading and examining the dataset, extracting drug interaction pairs, and preparing a small local lookup database (such as a JSON file).

To implement the DDI feature, so far, I created three versions of the lookup logic. In the first version, I attempted to convert each drug name into an RXCUI through RxNorm, but the resulting JSON file contained almost all null values.

This happened because the Kaggle dataset drug names do not always match RxNorm naming rules. RxNorm's fuzzy search helps, but it still fails for multi ingredient drugs, non US brand names, obsolete drugs, spelling inconsistencies, and internationally sourced drug names. Additionally, the Kaggle dataset contains only generic names, so most lookups failed. My conclusion at this stage was that the drug names in the Kaggle CSV are real medications, many oncology or chemotherapy agents, but they are not fully aligned with U.S. specific RxNorm terminology.

My second version used a direct string to string search instead of RXCUI lookup. The user enters a drug name, and the system returns the matching interaction pairs from the dataset. This implementation worked successfully, and I verified it by testing several real examples from the dataset.

One important observation is that RxNorm operates primarily on generic names. To support brand name search, an automatic mapping between brand names and their corresponding generic names is required. I implemented this feature, but during testing I encountered a new issue, the backend started freezing.

When I tried running the backend, everything appeared to start normally, but the OpenAPI schema (/openapi.json) never loaded, and Swagger UI stayed stuck on "Loading…". The server didn't crash, no errors appeared in the terminal, and even the browser console showed nothing unusual. The request simply never completed. This indicated that FastAPI was freezing while generating the OpenAPI schema. Since the schema powers the entire interactive documentation, any unresolved import, circular reference, or problematic model can silently block schema generation. I began debugging from this point to identify which part of the new brand generic mapping code caused FastAPI to hang.

- Documenting my progress and challenges (2 hours):

At each step, I carefully documented my work so I could track my progress, share updates with my teammates, and refer back to it during the next stages of development.

1.2 Lana

For this sprint, my goal was to integrate much of the backend APIs (FHIR, RxNorm, and PubMed) with the frontend and to display what these connections can do for users interacting with the website. Much of my work this sprint went into planning, designing, and implementing a website for both providers and patients. I completed the following work:

- Planning & Research phase: implementation plan / ideas for frontend (5.5 hours):
    - Researched more into how TypeScript connects backend APIs into the frontend.
    - Planning the best way to configure the patient and provider interfaces for ease of use and the most intuitive way to navigate the website based on similar portals such as MyChart, Athena, etc.
    - Created the following implementation plan:
        - 1. Create a centralized API service layer (defines all API-related TypeScript types used by the frontend, enforces the data exchanged between the frontend and backend)
        - 2. Create separate modules for each API: fhirApi.ts, rxnormApi.ts, pubmedApi.ts
        - 3. Identify provider dashboard features:
            - Patient search with filters (name, birth, date, gender)
            - Patient list with selection
            - Patient detailed view showing:
                - Medications with RxNorm lookup
                - Conditions
                - Pregnancy observations
                - Summary view
            - Medication safety checker:
                - Drug search with autocomplete
                - Lookup RxCUI

        - PubMed results showing affiliated articles
        - Display PubMed research findings (safe/unsafe, consult doctor)
- 4. Identify Patient portal features:
    - View own medications (if patientID is linked)
    - Display medication safety information
    - View personal pregnancy-related observations
    - Simple medication lookup/search + associated PubMed research
- 5. Define UI components to build:
    - PatientSearch.tsx – Search and filter patients
    - PatientList.tsx – Display search results
    - PatientDetail.tsx – Patient information view
    - MedicationList.tsx – Display medications
    - DrugSearch.tsx – Drug autocomplete and lookup
    - PubMedResults.tsx – Display research publications
    - MedicationSafetyCard.tsx – Safety information display
- 6. Create integration approach
    - Use existing getAuthHeaders() from utils/auth.ts
    - Create reusable hooks: usePatientSearch, useMedications, useDrugSearch, usePubMedSearch
    - Add loading states and error handling
    - Use React state management for selected patient/drug
- 7. Researching Technical considerations:
    - API base URL: Use proxy in development (vite.config.ts), environment variable in production
    - Authentication: Include bearer token in all requests
    - Error handling: Network errors, API errors, empty states

            - Loading states: skeletons or spinners
            - Caching: consider React Query or SWR for API caching
            - Type safety: Define TypeScript interfaces for all API responses
- Implementation phase – integration of RxNorm, PubMed, FHIR APIs (13 hours):
    - 1. Frontend API Service Layer
        - Implemented RxNorm API Client (services/rxnormApi.ts):
            - autocompleteDrugs() – drug name autocomplete suggestions
            - lookupRxCui() – find RxCUI for drug names
            - getRxNormProperties() – get detailed drug properties
        - PubMed API Client (services/pubmedApi.ts):
            - searchPublications() – search pubMed for research publications
            - getPublicationDetails() – get detailed publication information
        - FHIR API Client (services/fhirApi.ts):
            - searchPatients() – Search for patients by name, DOB, or gender
            - getPatientMedications() – Get patient's medication list
            - getPatientConditions() – get patient's conditions
            - getPregnancyObservations() – get pregnancy-related observations
            - getPatientSummary() – get comprehensive patient summary
    - 2. TypeScript Type Definitions (types/api.ts)
        - RxNorm Types:

        - RxNormLookupResult – drug lookup response structure
    - PubMed Types:
        - Publication – individual publication data structure
        - PubMedSearchResponse – Search results with IDs
        - PubMedDetailsResponse – Detailed results with pagination
    - FHIR Types:
        - Patient – Patient data structure
        - PatientSummary – Comprehensive patient summary structure
    - Base Types: APIResponse<T> – Standardized API response wrapper
  - 3. UI Components
    - Medication Components
        - DrugSearch.tsx – Drug search and lookup
        - MedicationSafetyCard.tsx – Medication safety assessment display
        - MedicationList.tsx – Patient medication list component
    - Provider Components
        - PatientSearch.tsx – patient search
        - PatientList.tsx – patient list display component
        - PatientDetail.tsx – comprehensive patient information view
    - Application Components
        - ProviderApp.tsx – provider portal
        - PatientApp.tsx – patient portal
  - 4. Styling and UX
    - Consistent design system, responsive design, loading states, error handling, empty states, hover effects, smooth animations, color coding, accessibility

    - 5. Architecture
        - Component-based, service layer, type safety, state management, error boundaries, API abstraction
- Merging pull requests / reviewing code for the team (2 hours)
- Sprint #5 Document (1 hour)

1.3 Valentina

My goal for this sprint was to migrate the FHIR client to async architecture, implement resource deletion operations with cascade support, add native FHIR validation, create an audit logging system, improve Synthea data generation, implement patient ingestion tracking, and deploy the backend. Over approximately 30+ hours, I completed the following work:

- Async/Await Architecture Migration (4 hours):
    - Migrated the entire FHIR client service from synchronous to fully async operations using `AsyncFHIRClient` from fhir-py, for better performance and scalability
    - Converted all API handlers to async pattern and updated test infrastructure to use `asyncio.run()` for async test execution
- Resource Deletion Operations (3 hours):
    - Implemented comprehensive delete functionality for all FHIR resource types, and added tests
    - Added cascade delete support for patients with automatic cleanup of referencing resources (e.g. MedicationRequests, Conditions)
    - Created new API endpoints to delete a patient_id and delete a resource (by resource type and resource id)
- Native FHIR Validation (2 hours):
    - Integrated fhir-py's native validation using resource.is_valid() method, which can be toggled with validate parameter
    - Validation performed before resource creation/update operations
    - Implemented fallback mechanism that continues with resource operations even if validation fails (advisory mode)
- FHIR Operations Audit Logger (2.5 hours):

    - Created a comprehensive audit logging system for all FHIR operations. Log entries include timestamp, operation type, resource details, success status, and error messages
    - Implemented persistent JSON log files stored in `backend/logs/` directory with timestamped filenames
    - Integrated throughout FHIR client for debugging, compliance, and monitoring
    - Created scripts/view_fhir_audit_logs.py utility to view and filter audit logs
- Synthea Data Generation Improvements (2 hours):
    - Improved Synthea modules for realistic pregnancy data: lactation_status.json, tracks lactation period after live birth (24-40 weeks); gestational_weeks.json, attribute-driven gestational week tracking; enhanced pregnancy_exposures.json, Improved trimester-timed medication exposures
    - Added .env file loading support in Synthea run script for better configuration management
    - Generated new improved synthetic data
- Local HAPI FHIR Server Integration (1 hour):
    - Enabled HAPI FHIR server in docker-compose.yml for local development
    - Configured with health checks and persistent volume storage (hapi-data). Server accessible at http://localhost:8080 (configurable via HAPI_FHIR_PORT)
- Deployment Platform Research and Evaluation (1 hour):
    - Researched and evaluated multiple deployment platforms for FHIR server and backend hosting
    - Reviewed Fly.io, Render, Railway, Netlify, Vercel, and Supabase capabilities and limitations
    - Analyzed Docker container support, memory requirements, pricing, and deployment constraints

    - Determined that Vercel supports FastAPI serverless but not Docker containers, limiting HAPI FHIR deployment options
    - Selected Fly.io for backend deployment based on Docker support, flexible memory configuration, and cost-effectiveness
- Fly.io Deployment and Platform Migration (6 hours):
    - Deployment for backend and HAPI FHIR JPA server starter Docker image (hapifhir/hapi-fhir-jpaserver-starter)
    - Initially attempted deployment to Render but encountered critical port configuration issues (see challenges section), so migrated from Render to Fly.io
    - Successfully deployed FastAPI backend Docker image to Fly.io (frontend already deployed on Vercel)
        - Created fly-hapi.toml and fly-backend.toml configuration files for both services
    - Created short-cuts in Makefile for health checks. Initially for Render but later migrated to Fly.io
    - Configured H2 database with persistent volumes for data persistence
    - Enhanced CORS configuration for multiple origins (local, Vercel, custom domain)
    - Note: HAPI FHIR server deployment remains challenging; evaluating alternative approaches including using public HAPI FHIR server with identifier-based filtering (see challenges section)
- Patient Ingestion Tracking System (3.5 hours):
    - Implemented custom identifier system (\`https://pregnancy-med-checker.org/identifier/ingested-by\`) to track patients ingested by this application
    - Added automatic tagging of all ingested patients with custom identifier during ingestion workflow
    - Enables data isolation when using public FHIR servers like while maintaining clear separation of ingested data
- Enhanced Patient Search with Filtering (2 hours):

    - Added only_ingested_patients parameter to filter searches to only ingested patients
    - Modified search_patients() in FHIR client to automatically apply ingestion identifier filter
    - Updated FHIRClientPort protocol with new parameter signature
- Bulk Patient Management (1 hour):
    - Implemented delete_all_ingested_patients() method for bulk deletion with cascade support
    - Implemented get_ingested_patient_ids() method to list all ingested patients with count and IDs
    - Added corresponding API handlers with proper error handling, as well a commands in Makefile
- Code Quality & Architecture Improvements (1 hour):
    - Removed unused FHIR libraries
    - Created new FHIRResourceProcessor class for centralized resource preparation
    - Enhanced resource cleaning utilities, error handling, type hints and protocol definitions
    - Improved test infrastructure with better error handling in async operations
- Migration to Pytest (1 hour):
    - Migrated FHIR integration tests to use pytest, configured pytest in pyproject.toml, updated Makefile targets to use pytest commands.
    - Created scripts to delete all ingested, list ingested, and ingest all bundles within scripts/ folder instead of having it within tests
- Pregnancy and lactation stage extraction (2 hours):
    - Added three tests for pregnancy stage extraction, lactation stage extraction, and medications list retrieval, with corresponding test data bundles
- Lactation and pregnancy stage endpoints (1.5 hours):
    - Implemented helper methods to get a patient's lactation stage, pregnancy stage, and list of medications

    - Added corresponding handlers and API endpoints
- Synthea data improvements (1.5 hours):
    - Improved the custom modules to work with the Synthea pregnancy module and made edits to Synthea modules to be able to increase likelihood of pregnancy
    - Repackaged the JAR to only use selected and edited modules instead of all default modules.
- Sprint Documentation (1 hour):
    - Documented all accomplishments, challenges, and technical details for Sprint 5 document

2 CHALLENGES

2.1 Faezeh

Throughout the past two weeks, I encountered several technical and integration challenges while working on both the OpenFDA feature and the drug-drug interaction (DDI) feature. One of the earliest challenges arose during the OpenFDA integration, which required additional error handling logic and careful testing to make sure our backend remained stable. I also ran into a blocking issue with the python jose library used for JWT handling. This prevented the backend from starting until I researched the cause and resolved the configuration problem.

A major challenge involved identifying a suitable data source for drug-drug interactions. Although multiple datasets exist, many of them were not compatible with our project's requirements. For example, the SNAP dataset uses DrugBank identifiers rather than readable drug names, making it impractical for our lookup system. DrugBank provides high quality clinical data but requires an academic license that is still pending approval. Other datasets, such as Guide to Pharmacology, focus on drug-target interactions rather than drug-drug interactions. Even RxNav's interaction API has been discontinued. This required significant time evaluating each option, understanding its limitations, and determining what was realistically usable for the project.

The most challenging part of the DDI integration was attempting to map drug names from the Kaggle dataset to RXCUIs through RxNorm. Most lookups failed because the dataset includes non-U.S. or oncology specific drug names that do not align well with RxNorm terminology. Fuzzy search did not reliably resolve spelling variations, or international brand names. Thus, the first implementation produced almost entirely null RXCUI values. When I moved on to implementing automatic brand to generic mapping to improve lookup support, I encountered an unexpected backend freeze, that the OpenAPI schema (/openapi.json) never loaded, causing Swagger UI to remain stuck on "Loading…". The lack of visible errors made debugging more difficult, and the issue likely stemmed from imports or models interfering with schema generation. This required additional investigation before continuing development.

2.2 Lana

In this sprint, I faced a few issues during implementation. Some of the main challenges are noted below.

- Current research for ibuprofen / acetaminophen reflects the current administration's beliefs, so all of the pubMed results being scraped is from recent publications which can be biased and not exactly reflective of reliable and thorough research.
    - For the final sprint, I think it would be beneficial to identify a way to show publications based on their highest impact factor instead of showing only recent publications.
- Pulling patient info from fhirAPI vs. Synthea ingested patients:
    - I developed the frontend initially to use a patient name from the fhirAPI (Jane Doe), and there are 100+ "Jane Doe" patients in the public server. Now we are using the Synthea data which does not include this patient, and the backend is configured to only search for ingested patients (only_ingested_patients=True), so Jane Doe does not appear unless you search with only_ingested_patients=false. The patient names that are in the current Synthea data we have are also test accounts, such as

"Simple Test" or "Test Lactating", which is not ideal to use for this application. I am trying to decide whether I should just generate a patient named "Jane Doe" using synthea, or to allow searching all patients (not just the ingested data) in the frontend. I am still considering what the best approach would be and what makes most sense for our application use.

- Drug Search autocomplete is slow on the UI
    - RxNorm autocomplete feature introduces noticeably latency in the UI. When quickly typing a drug name, the autocomplete pauses and does not let users continue typing until it shows a dropdown of the potential drugs a user could be trying to search up. Results sometimes appear after a user has typed only a few characters, which could help quickly access the drug, but also could be showing the incorrect drug and slows down the process of getting the actual drug. A potential fix in the next sprint can be to cache common drug queries and to slow down how often autocomplete requests fire.

2.3 Valentina

- OutOfMemoryError on Fly.io (Critical Challenge): The HAPI FHIR server was crashing with java.lang.OutOfMemoryError: Java heap space during bulk operations and structure definition loading. The initial 1GB RAM allocation was insufficient.
- Health Check Timeouts: Health checks were failing during FastAPI startup because the grace period was only 10 seconds, which wasn't sufficient for the application to complete its startup event (including FHIR connection test). I increased both TCP and HTTP check grace periods from 10s to 30s in fly-backend.toml to allow adequate time for startup completion.
- Platform Migration Complexity and Ongoing HAPI FHIR Deployment Challenges:
    - Initially attempted to deploy to Render, but encountered critical issues with port configuration. Render's platform always assigned

port 10000 and this port could not be changed or configured, which conflicted with our application's expected port configuration. Additionally, Render's free tier (512MB) was insufficient for HAPI FHIR.
    - Migrating from Render to Fly.io required learning a new deployment platform, understanding Fly.io's configuration system, and adapting our Docker-based setup.
    - While I successfully deployed the FastAPI backend Docker image to Fly.io, deploying the HAPI FHIR JPA server starter Docker image (hapifhir/hapi-fhir-jpaserver-starter) proved challenging due to memory requirements and cost considerations.
    - So, instead of deploying our own HAPI FHIR server, I implemented a custom identifier system that automatically tags all patients ingested by our application. This provides sufficient data isolation when using the public server, eliminating the need for a separate FHIR server instance for demonstration purposes.
- Async Migration: Converting all functions to async/await pattern required careful attention to ensure proper async context management and error handling.
- Synthea modules: I needed to run Synthea with only my custom modules and exclude specific default modules, but when using the JAR the module-loading properties it didn't reliably disable the defaults, which kept loading and triggering events. So, I tried to clone the Synthea repository and run Developer Setup instead, but then I had SSL verification errors. Finally, I unzipped the Jar, changed the modules/ folder and repacked.

3 SPRINT PLANS

The next sprint is the final sprint, so the team will focus on getting the application fully functional and ready for final submission.

Valentina's focus for the final sprint: I will primarily focus on connecting the FHIR data into something usable from both the frontend and backend, and

connecting patient medication requests with drug interactions. This includes ensuring that patient data from FHIR can be properly retrieved and displayed in the frontend, and that medication requests can be checked against interaction databases. I will ingest all the Synthea data into the public HAPI FHIR server using the new identifier method, which will automatically tag all patients with our custom identifier for data isolation, and test operations with these patients using the public server. This will provide a complete dataset for the project and demonstration purposes. I will implement a helper method to easily ingest data for a demo, look into a database for persistent storage, and if time permits, look into deploying our own server instance again. I will also assist with whatever else is needed by the team to complete the project and ensure everything is functional and ready for the final submission.

For the next sprint, Faezeh will implement "Brand Name to Generic Name Mapping Debugging" for drug-drug interaction. RxNorm primarily operates using generic drug names. To support brand name searches, she implemented an automatic mapping feature between brand names and their corresponding generic names. To complete testing, the next step is to debug this issue systematically to identify and fix the part of the brand-to-generic mapping that is causing the freeze, ensuring smooth schema generation and proper backend functionality. Her next task will be "Drug–Drug Interaction (DDI) Module Improvements". This includes reviewing the current DDI implementation to identify potential optimizations, edge cases, or missing interactions. She is also tasked for "Final Report Completion", dedicating time to summarize all project work, including OpenFDA integration, RxNorm integration, and DDI features.

Lana's focus for the final sprint will be to add a data visualization component for pregnancy observations to allow for both the patient and the provider to visualize consistencies/fluctuations across trimesters and even across different pregnancies (if applicable). I also want to add additional pregnancy-related information for each patient, such as how many births they had or how many pre-term births. This was an idea given to me from a nurse, as this information can help them evaluate their pregnancy-risk, so this is definitely a great idea to incorporate into our application.

Lana also wants to incorporate a medication reminder/alert function for patients to be aware of when to take their medication or an alert if any of their medications interacts with each other.

Lana will also work to add a provider notes/comments function for the provider to write inside their portal, and a place for the patient to view the comments on their patient portal as well. This is helpful for the provider to remind themselves of any key details about the patient, and for the patient to be up to date with anything that the provider makes note of that is relevant to their health.

Another function that Lana will work to add is patient-provider messaging. I think this will really tie the provider and the patient portals together. I think this will really be effective in showing how the patient can rely on their provider during their pregnancies and be able to ask questions and communicate directly with their provider through this messaging system.